

kubemend

A Kubernetes remediation agent whose only
write surface is a git commit

Srivatsa Kamballa

github.com/Srivatsa03/kubemend

Version 0.1.0 (alpha) • Apache-2.0 • August 2026

Abstract. Autonomous remediation for Kubernetes is limited less by diagnostic quality than by trust. A tool that can identify a crash loop is straightforward to build; a tool an operator will permit to act on production unattended is not, and the obstacle is that no convincing answer exists to the question of what prevents it doing something catastrophic without supervision. KUBEMEND is an attempt at that answer, constructed so that the safety properties are structural rather than behavioural. The agent holds no cluster credentials and issues no write to the Kubernetes API. It reads cluster state with read-only verbs, selects from a closed set of six typed actions each carrying the state it found and the state it intends, subjects the resulting plan to a pure-function policy gate, and emits the change as a commit to a GitOps repository, which an existing reconciler carries to the cluster. Undoing the agent is `git revert`. After committing, it verifies: it polls the workload, requires two consecutive clean observations before declaring recovery, and withdraws its own commit when presented with positive evidence that the fix did not hold. Every run is recorded to an append-only incident log, which yields the measurement the field generally omits — the agent’s revert rate, its own accuracy on real outcomes. On a live k3d cluster the system demonstrates both terminal outcomes in a single run: one fix recovered in 26 seconds and was kept, and a second, applied against two consecutive bad releases, was still failing after 76 seconds and was reverted automatically. The implementation is 3,248 lines of Python with no runtime dependencies, covered by 157 tests, with continuous integration exercising the full loop against a real cluster on every push.

Contents

1	The problem is trust, not detection	2
1.1	The central design decision	2
2	Architecture	2
3	Detection	3
4	Typed actions	3
4.1	One plan per incident, and the freedom to abstain	4
5	The policy gate	4
5.1	Three levels of autonomy, granted per action	4
6	Writing the fix	5
6.1	Rollback is a git operation, not a computed edit	5
6.2	The diff is one line	5
6.3	It refuses more than it writes	5
6.4	The commit message is the evidence	6
7	Verification, and the discipline of undoing	6
8	The incident log	7
9	The console	8
9.1	An instructive bug	8
10	Evaluation	9
10.1	Offline, against a recorded cluster	9
10.2	Live, on a real cluster	9
10.3	Implementation and test surface	10
11	Limitations and honest scope	10
12	Reproducing this work	11
A	Incident log schema	11
B	Command reference	11

1 The problem is trust, not detection

Every observability vendor can tell an operator that a pod is in `CrashLoopBackOff`. Very few of their agents are permitted to do anything about it, and the reason is not model quality. It is that granting write access to a production cluster requires an answer to a specific question — what stops this from doing something catastrophic at three in the morning — and “the model is usually careful” is not an answer. It is a statement about average behaviour offered in response to a question about worst-case behaviour.

The gap has a structural cause. An agent that emits shell commands, `kubectl` invocations, or freely-generated manifests has an unbounded action space. Three consequences follow immediately, and none of them are implementation defects that a better model would resolve:

- The set of things it might do cannot be enumerated, so it cannot be reviewed in advance.
- The blast radius of a proposed change cannot be computed before it runs.
- There is no mechanical inverse, so recovery from a bad action is itself an act of judgement under pressure.

Every safety property an operator would want is therefore unavailable *in principle*, not merely unimplemented. KUBEMEND starts from the opposite constraint: restrict the action space until those properties become computable, then let the agent diagnose as freely as it likes inside it. The design separates the two concerns deliberately — **diagnose broadly, act narrowly** — and every constraint on the acting half is code with tests rather than an instruction in a prompt.

1.1 The central design decision

KUBEMEND has no cluster credentials capable of mutation and issues no write to the Kubernetes API. Its only write surface is a commit to the GitOps repository that already defines the cluster. An existing reconciler — Argo CD, Flux, or whatever the team already runs — carries the change the rest of the way.

This single choice inherits an entire safety apparatus rather than reimplementing it:

- **An audit trail** exists because git is one, with authorship, timestamps and message.
- **Review before rollout** is available because a pull request is the native unit of review.
- **Rollback** is `git revert`, a command every operator on the team already knows at 3am.
- **Reconciliation** is someone else’s well-tested problem.

It also imposes a real constraint, which is the point: an action that cannot be expressed as an edit to a manifest cannot be performed at all. A rolling restart has no manifest representation, so KUBEMEND refuses it with a reason rather than approximating it with an annotation nobody asked for.

2 Architecture

The loop is six stages, each a pure function of the previous stage’s output wherever it can be. Purity is not stylistic here: it is what allows the entire decision path to be tested against recorded cluster state, with no cluster present and no mocking of the parts that matter.

1. **Read.** Cluster state via read-only verbs on the existing kubeconfig, or a recorded JSON snapshot.
2. **Detect.** Eight rules produce typed `Finding` objects with severity and evidence.
3. **Plan.** Findings are grouped per workload; each incident yields at most one plan.
4. **Gate.** A pure function returns a verdict: refused, or permitted at a specific autonomy level.
5. **Emit.** The plan is rendered as a surgical manifest edit and committed, or opened as a pull request.

6. **Verify.** The workload is polled; a fix that does not hold is withdrawn by `git revert`.

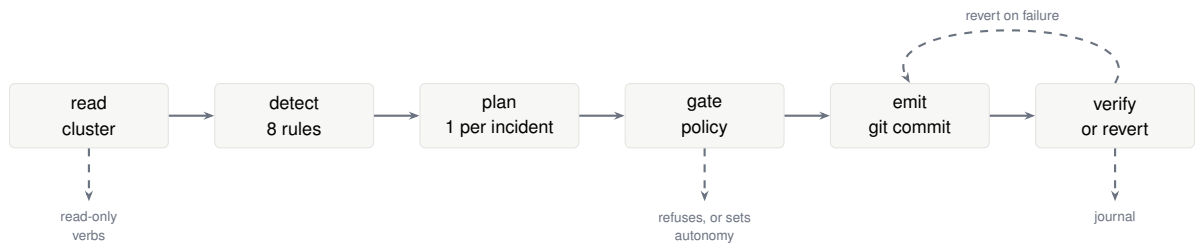


Figure 1: The remediation loop. Everything left of *emit* is a pure function of recorded cluster state and therefore testable without a cluster.

Every stage writes to an append-only incident log (§8), which is what makes the agent’s own accuracy measurable rather than asserted.

3 Detection

Detection is deliberately the least constrained part of the system, because being wrong here costs a false report rather than a bad change. Eight rules run over cluster JSON and emit findings carrying rule, severity, target and structured evidence.

Rule	Severity	What it catches
crashloop	critical	Container the kubelet has given up restarting promptly
oomkilled	critical	Killed by the kernel for exceeding its memory limit
image_pull	critical	Bad tag, bad registry, or missing pull credentials
config_error	critical	References a ConfigMap or Secret that does not exist
rollout_stuck	critical	Rollout past its progress deadline
replica_shortfall	critical / warning	Fewer replicas available than desired
unschedulable	warning	No node can fit the pod
flapping	warning	Restarting repeatedly while still reporting healthy

Table 1: The detection rules. Severity describes how bad the observed condition is, independent of confidence in any fix.

`flapping` is the rule worth singling out. The pod is up, dashboards are green, nothing pages, and the container has died eleven times today. That is the failure mode most likely to survive unnoticed for weeks, and it is invisible to any check that asks only whether pods are running.

4 Typed actions

KUBEMEND never emits commands. It selects from a closed set of six typed actions, and every action carries both the state it found (*before*) and the state it intends (*after*).

Three properties follow directly from the type, not from the agent’s care:

- **Reversible by construction.** An action holds *before* and *after*, so its inverse is a field swap. An action whose *before* could not be captured is refused up front, rather than discovered to be irreversible at rollback time.
- **Blast radius computable before execution.** Every action declares the pods it disrupts, so a plan is measured and refused while it is still text.
- **Reviewable.** A typed action renders to a deterministic diff. A reviewer sees a bounded change, not a prompt’s output.

Action	Effect	Manifest field
scale	Change replica count	<code>spec.replicas</code>
rollback	Revert a workload to a prior revision	restored from git history
restart	Trigger a rolling restart	none — refused on emission
set_resources	Adjust requests or limits	<code>...containers.c.resources</code>
set_image	Pin or correct a container image	<code>...containers.c.image</code>
set_probe	Adjust probe timings or thresholds	none — refused on emission

Table 2: The closed action set. Each entry is a change an experienced operator makes routinely, whose effect is local and whose reversal is mechanical.

Adding a kind to this set is a security decision rather than a feature decision, because it widens what an autonomous process may do to a live cluster.

4.1 One plan per incident, and the freedom to abstain

An incident is something that happened to a service, and responders handle them one at a time. Batching unrelated problems into a single change would trip every blast-radius limit simultaneously and couple the fate of unrelated services. Separate plans are separately reviewable, applicable and revertible.

Abstention is treated as a correct outcome rather than a failure to be minimised. On the shipped fixture — a recorded snapshot of a cluster having a bad afternoon — **13 findings produce 4 plans, and 5 findings produce no action at all**. A missing ConfigMap needs a value the agent has no business inventing. An unschedulable pod is a capacity decision. A container restarting for unclear reasons needs a human to read the logs. Reporting these clearly and stopping is the correct behaviour, and the test suite asserts it.

5 The policy gate

The gate is a pure function from (plan, policy) to verdict, with no model in the loop. It enforces six properties.

1. **Protected namespaces are excluded outright.** `kube-system`, `argocd`, `flux-system` and similar are refused rather than guarded by a threshold. Touching them can remove the machinery that would let you recover.
2. **Only action kinds named in policy**, with everything else denied by default.
3. **Blast radius bounded** on pods, workloads and namespaces — for the plan as a whole, since three individually harmless restarts are not a harmless change.
4. **No action without a computable undo.**
5. **Flap protection.** A cluster that has already needed several fixes this window is a human’s problem, not a loop’s.
6. **An unconfigured `Policy()` permits nothing.** A permissive default is the difference between a bounded assistant and an unsupervised process with cluster credentials.

Policy is data, not code, so the rules governing an autonomous system are themselves reviewable, diffable and revertible — the same property the agent’s own actions are required to have.

5.1 Three levels of autonomy, granted per action

Trust is granted per action class and per environment rather than assumed globally, mirroring the crawl, walk, run progression that operations teams already use.

Level	What happens
report	Findings only. Nothing is proposed.
propose	A branch and a pull request, for human review.
apply	Committed to the GitOps branch unattended.

Under the shipped conservative policy only `rollback` may apply unattended, because it moves a workload to a state that demonstrably ran in this cluster and its inverse is exact. Everything else waits for a human.

A plan’s autonomy is the minimum across its actions. One action requiring review holds back the whole plan, because applying the safe half of a plan that was reasoned about as a unit is frequently worse than applying none of it.

Workload	Action	Verdict	Reason
payments/checkout	rollback	apply	within all limits
jobs/report-worker	rollback	apply	within all limits
payments/api	set_resources	propose	autonomy_ceiling
kube-system/coredns	rollback	refused	protected_namespace

Table 3: Gate verdicts on the shipped fixture under the conservative policy. The identical `rollback` action is applied on one workload and refused on another purely on namespace, which is the gate doing its job.

6 Writing the fix

6.1 Rollback is a git operation, not a computed edit

In a GitOps repository the manifest is the source of truth, so returning a workload to its previous revision means restoring the file to its previous committed state. Nothing is reconstructed — the prior version is present in the history, byte for byte, comments and all. This falls out of the architecture rather than being engineered, and it is the strongest single argument for routing an agent’s actions through git.

6.2 The diff is one line

The obvious way to edit a manifest is to parse the YAML, mutate the object, and serialise it back. That produces a correct file and a useless commit: the serialiser rewrites the whole document, drops comments, reorders keys and normalises quoting, so a reviewer sees three hundred changed lines when the change was one number.

The entire value of routing actions through git is that a human can read the diff before it reaches the cluster. A commit that reformats the file is unreviewable, which defeats the purpose. KUBEMEND therefore performs *surgical* text edits against an indentation-path tracker rather than parsing and dumping. Trailing comments survive, including the exact whitespace preceding them.

Listing 1: An emitted change. One line, comment and spacing intact.

```
-      image: nginx:1.27-alpine-typo    # known good
+      image: nginx:1.27-alpine      # known good
```

6.3 It refuses more than it writes

- **A workload with no manifest** is reported, never created.
- **A value that drifted** — the repository says something other than what the cluster reported — is left alone. Somebody changed it, and their change is not the agent’s to discard.

- **A field that does not exist** is not added. Inventing a resource limit the author never wrote is a different class of change from adjusting one that is there.
- **An action with no manifest representation**, such as a rolling restart, is refused with a reason rather than approximated.
- **A dirty working tree** blocks emission entirely, so an agent’s commit never absorbs somebody’s in-progress edit.

6.4 The commit message is the evidence

Every commit carries what produced it, and closes with the instruction that matters.

Listing 2: A generated commit message, verbatim from the live demo.

```
rollback payments/deployment/checkout

Observed:
  [critical] cannot pull image nginx:1.27-alpine-typo for
  checkout-6d78fb9788-t488h

Changed:
  restored clusters/prod/checkout.yaml to f14c37f1
  reason: image_pull: cannot pull image nginx:1.27-alpine-typo

Blast radius: 2 pod(s) across 1 workload(s).
Undo: git revert this commit.
```

Plans held at `propose` push their branch and open a pull request whose body is this message. A repository with no remote, or a missing `gh`, is reported rather than treated as an error, since that is the normal case for a local checkout.

7 Verification, and the discipline of undoing

A remediation loop that stops at “committed” is half a loop. The agent has acted on a diagnosis that may have been wrong, and until something checks, the cluster is in a state nobody has confirmed is better than the one it replaced.

Recovery is defined narrowly: **the findings that motivated the change are gone, and no new critical finding has appeared on that workload**. Not “the workload is perfect” — a service can carry a warning for weeks — and not “the pods are running”, which is true moments before a crash loop starts.

A workload must read clean **twice consecutively**, because a rollout looks briefly healthy as it begins. A `settle` grace period precedes the first check, since a reconciler needs a moment to apply the commit and checking immediately would reliably observe the old state and call it a failure.

Outcome	Meaning	Reverts?
recovered	the motivating findings are gone	no
still_failing	they remain, or a new critical one appeared	yes
indeterminate	the cluster could not be read	no

The third row is the one worth arguing about, and it encodes two decisions that matter more than the polling mechanics.

Silence is not success. A poll that cannot reach the cluster proves nothing, so it never counts toward recovery — and, more subtly, it *breaks the recovery streak* rather than being skipped. Two clean reads either side of a blind one are not two consecutive observations, and allowing silence to bridge them would contradict the rule the module is built on. This was originally implemented the other way; a test

asserted the correct behaviour and the implementation was changed to match it, on the grounds that the module's own stated contract was the authority.

Indeterminate does not trigger a revert. Reverting on positive evidence of continued failure is right. Reverting because the API server was briefly unreachable would undo a fix that may have worked, on no evidence at all, and convert a network blip into a second production change. An unverifiable outcome therefore stops and asks for a human.

When it does revert, it uses `git revert` rather than a reset. The history of an automated system acting on production is worth keeping, and a reviewer should see both that it acted and that it withdrew.

8 The incident log

A single run answers “what is wrong now”. The questions that decide whether an automated system deserves more autonomy are all longitudinal, and none of them are answerable from standard output. Every run therefore records what it saw, decided, wrote and confirmed to an append-only SQLite file at `~/.kubemend/journal.db`.

Two numbers come out of it that were previously unavailable.

Revert rate is the agent's own accuracy — how often its fix failed to hold and it withdrew its own commit. It is the only honest input to the question “should this be allowed to `apply` rather than `propose`?”, and a tool that quietly declined to measure it would be asking for trust it had not earned.

Repeat offenders are the workloads a rollback keeps papering over. Something that crash-loops every week does not have a bad release, it has a bug, and the log is what makes that visible instead of letting three successful rollbacks look like three successes.

Refusals are recorded alongside changes: what the gate declined to touch, and why, is as much a part of the audit trail as what it wrote.

Two engineering choices are worth stating. Storage is standard-library `sqlite3`, so the zero-dependency property holds and the log remains a single file that can be copied, inspected with any tool, or deleted. And **bookkeeping never breaks remediation**: every write is wrapped, an unwritable journal degrades to a note in the output, and the run continues. Failing to fix a broken cluster because a log file was read-only would be a poor trade.

Listing 3: `kubemend log`, from the live demo run.

```
incident log   /tmp/kubemend-demo-journal.db

2026-08-15 16:46:25  reverted      payments/deployment/checkout
                    cannot pull image nginx:1.27-broken-b for checkout-598f76497b-tmtwg
2026-08-15 16:44:46  verified      payments/deployment/checkout
                    cannot pull image nginx:1.27-alpine-typo for checkout-6d78fb9788-449mb

totals    across 2 run(s)

incidents    2
committed    2
refused      0
verified     1
revert rate  50%  (1 of 2 fixes did not hold)

keeps coming back  a rollback is not going to fix these

2x  payments/checkout
```


9 The console

`kubemend log` answers a question the operator already knew to ask. The console serves the other mode: scanning a week of automated changes to production and stopping at the one that looks wrong. Each incident opens to the full audit trail — the evidence observed, the action selected with *both* its before and after, the commit, the verification outcome, and the revert if there was one.

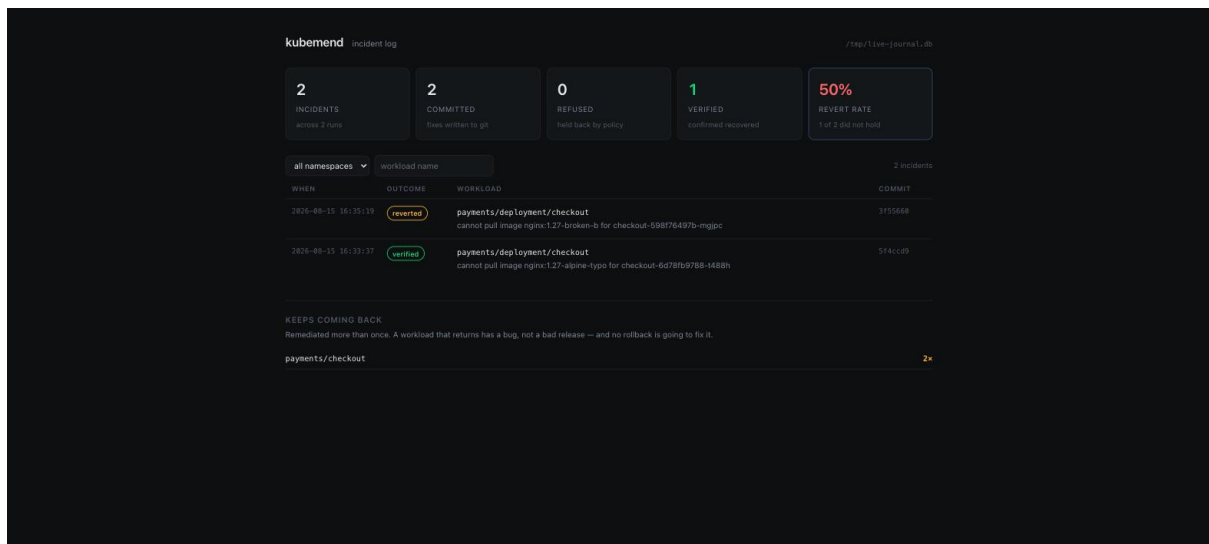


Figure 2: The console, showing real output from the live demo: two incidents on one workload, one fix that held and one that did not.

Three constraints shaped it, and each was a choice rather than an accident.

- **Read-only, structurally rather than by policy.** There are no write routes at all, so nothing reachable from a browser can touch a cluster or a repository. The only write surface in this project is a git commit, and a console with a “remediate now” button would quietly make that claim untrue. `POST`, `PUT`, `DELETE` and `PATCH` are refused explicitly and covered by tests.
- **Localhost by default.** The journal is an inventory of what is fragile in a cluster — workloads, images, namespaces, failure modes. A `-host` flag exists for people who mean it, and the server says so on startup.
- **Standard library only.** `http.server` over the same SQLite file. Installing a web framework in order to look at a local database would be a poor reason to break the dependency promise.

9.1 An instructive bug

The console initially shared one SQLite connection across the threaded server. SQLite connections are bound to their creating thread, so every request after the first failed — and failed *silently*, because the journal’s read path swallows errors by design so that bookkeeping cannot break remediation. The symptom was an empty list rendered as “no incidents”, which is a wrong answer disguised as a valid one.

The fix is a connection per request, which is cheap for SQLite and has the welcome side effect that a console reading a file a `remediate` run is still appending to shows what was just written. The regression test drives a real socket rather than calling the router in-process, because an in-process call would have passed happily against the broken design; this was confirmed by reproducing the failure before trusting the test.

10 Evaluation

10.1 Offline, against a recorded cluster

The repository ships a snapshot of a cluster with several simultaneous problems, so the whole analysis path runs with nothing installed.

```
$ kubemend diagnose --snapshot fixtures/broken-cluster.json

kubemend    13 finding(s), 4 incident(s), policy 'conservative'
```

Thirteen findings become four plans; five findings produce no action at all. Of the four plans, one is refused outright for touching a protected namespace, one is held at propose, and two are permitted to apply.

10.2 Live, on a real cluster

`demo/run.sh` stands up a k3d cluster and a git repository holding its manifests, applies the repository the way Argo CD or Flux would, ships a release that breaks, and lets KUBEMEND read the cluster and write the fix back. The committed transcript in `demo/transcript.txt` is the verbatim output of a real run.

The demo proves *both* terminal outcomes, which matters more for an unattended system than proving the happy path twice.

A fix that works. A bad image tag is shipped. KUBEMEND detects the pull failure, restores the manifest from git history, commits, and watches.

```
✓ APPLY    payments/deployment/checkout
           restored clusters/prod/checkout.yaml to f14c37f1
           commit 9465b30a on main
           -      image: nginx:1.27-alpine-typo    # known good
           +      image: nginx:1.27-alpine        # known good
           watching for recovery...
           recovered after 26s
           1 commit(s) written.
```

A fix that does not. The demo then ships *two* bad releases in a row, so that rolling back one still leaves the workload broken. KUBEMEND commits its rollback, fails to observe recovery, and withdraws its own change.

```
✓ APPLY    payments/deployment/checkout
           restored clusters/prod/checkout.yaml to 92a6903d
           commit 9b07e933 on main
           watching for recovery...
           still failing after 76s: cannot pull image nginx:1.27-broken-a
           reverted in 0b0d81f4
           0 commit(s) written.

c2b3faa Revert "rollback payments/deployment/checkout"
9b07e93 rollback payments/deployment/checkout
9d637fb bad release B
```

The repository ends the run in the state a human last approved. The agent never called the Kubernetes API to change anything: it read the cluster, committed a fix, watched, kept the one that worked and reverted the one that did not.

Read separately these are two anecdotes. The incident log turns them into the measurement the project is actually arguing about: **a 50% revert rate, computed from real outcomes rather than**

Scenario	Outcome	Time to verdict	Action taken
One bad release	recovered	26 s	commit kept
Two bad releases	still_failing	76 s	commit reverted automatically

Table 4: Live k3d results, from the committed transcript. Wall-clock timings vary between runs with cluster scheduling; the outcomes do not.

claimed, together with `payments/checkout` flagged as a repeat offender because it was remediated twice.

10.3 Implementation and test surface

Module	LOC	Test file	Tests
<code>signals.py</code>	382	<code>test_signals.py</code>	23
<code>plan.py</code>	184	<code>test_plan.py</code>	25
<code>safety.py</code>	278	<code>test_safety.py</code>	28
<code>gitops.py</code>	430	<code>test_gitops.py</code>	33
<code>manifest.py</code>	204	(covered via gitops)	—
<code>verify.py</code>	171	<code>test_verify.py</code>	9
<code>journal.py</code>	435	<code>test_journal.py</code>	16
<code>serve.py</code>	397	<code>test_serve.py</code>	15
<code>cli.py</code>	536	<code>test_cli_log.py</code>	8
<code>model.py</code>	224	—	—
Total	3,248		157

Table 5: Python 3.10+, no runtime dependencies. A clean-environment install reports only kubemend itself.

Two testing decisions are load-bearing. The GitOps tests run against a *real temporary git repository* rather than mocks, because the behaviour under test is what git actually does with history and branches. And the verification tests inject a fake clock, which is what allows a 180-second timeout to be exercised in milliseconds — and which surfaced a genuine infinite-loop defect when a no-op sleep left the clock stalled.

Continuous integration runs the test matrix on Python 3.10 and 3.13, plus a separate job that installs k3d, creates a real cluster, and runs the full demo end to end on every push. A project whose claim is that it works against a real cluster should not prove it with mocks.

11 Limitations and honest scope

- **The planner is deterministic. There is no language model in the decision path.** This is sequencing rather than limitation: the parts that must be trustworthy are code with tests, so that when a model is introduced it correlates findings and writes narrative rather than deciding what happens to a cluster. It should not be described as an LLM-powered agent, because it is not one.
- **Version 0.1.0, alpha.** Not published to a package index.
- **Workload coverage is Deployments.** StatefulSets, DaemonSets and Jobs are not yet handled.
- **Signal coverage is pod- and workload-level.** Node-level, networking and storage signals are absent.
- **Verification is single-workload.** It confirms that the treated workload recovered, not that the change was harmless to its dependents.
- **The reconciler is assumed.** The demo substitutes `kubectl apply` for Argo CD or Flux; the agent’s contract ends at the commit.

The revert rate reported here (50%) comes from a demo deliberately constructed so that half its scenarios are unfixable by rollback. It demonstrates that the measurement works; it is not a claim about accuracy in production, and it should never be quoted as one.

12 Reproducing this work

Listing 4: Requires Python 3.10+, and for the live demo, k3d, kubectl and a Docker daemon.

```
git clone https://github.com/Srivatsa03/kubemend && cd kubemend
pip install -e ".[dev]"

# offline: the whole analysis path, no cluster required
kubemend diagnose --snapshot fixtures/broken-cluster.json
kubemend policy

# the full loop against a throwaway cluster, both outcomes
demo/run.sh

# what it knows about itself
kubemend log
kubemend serve          # http://127.0.0.1:8420

pytest -q               # 157 tests
```

A Incident log schema

Listing 5: Four tables. `run_id` is nullable: an incident recorded outside a run is still worth keeping.

```
runs      (id, started_at, policy, namespace, source, dry_run,
          findings, incidents, commits)

incidents (id, run_id?, created_at, namespace, kind, name, rule, summary,
          allowed, autonomy, refusals, commit_sha, branch, pr_url,
          verification, verify_detail, reverted_sha)

findings  (id, incident_id, rule, severity, summary, evidence)

actions   (id, incident_id, kind, container, before_state, after_state,
          impacted_pods, written, detail, skipped_reason)
```

An incident's state is derived rather than stored, so it cannot drift from the facts it is computed from: reverted if a revert SHA exists, else refused if the gate denied it, else reported if nothing was committed, else verified if verification confirmed recovery, else the verification outcome, else committed.

B Command reference

Command	Purpose
<code>kubemend diagnose</code>	Find problems and show the gated plan
<code>kubemend remediate</code>	Run the loop; <code>-repo</code> , <code>-verify</code> , <code>-pr</code> , <code>-dry-run</code>
<code>kubemend snapshot</code>	Record cluster state for offline analysis
<code>kubemend policy</code>	Print what each shipped policy permits
<code>kubemend log</code>	Read the incident log
<code>kubemend serve</code>	Browse the incident log in a browser

